

A Novel Method for Video Manipulation and Compilation Using Big Data Technologies

Abhishek V. Sagarnal

Dept. of CSE

PES University

Bangalore, India

sagarnalabhishek@gmail.com

Pradyun P. Rao

Dept. of CSE

PES University

Bangalore, India

pradyunrc@gmail.com

Sarthak S. Kumar

Dept. of CSE

PES University

Bangalore, India

sarthak.kumar@email.com

Divya D.

Dept. of CSE

PES University

Bangalore, India

divyadaya1005@gmail.com

Chandrashekhar Pomu Chavan

Dept. of CSE

PES University

Bangalore, India

cpchavan@pes.edu

Abstract—With the rise of multimedia applications, the demand for faster and scalable video processing systems has grown significantly. Traditional video processing methods face challenges such as high computational complexity, difficulty in scaling, and real-time performance limitations. This paper explores a solution that combines Apache Spark’s distributed computing capabilities with GPU acceleration using CUDA to improve video processing efficiency. By leveraging the Spark partitioning model alongside GPU parallelism, we enhance key tasks like frame extraction, dynamic filtering, and object detection. Experimental results show that this approach improves processing speed by 40% compared to CPU-based methods. This research provides a practical solution for large-scale, real-time video processing with a focus on entertainment, surveillance, and media analytics industries, while also discussing ways to further optimize large video data handling.

Index Terms—Big Data, Video Processing, Distributed Systems, Apache Spark, Apache Hadoop, Frame Extraction, PySpark, HDFS, CUDA, Scene Detection, Video Rendering, Scalability, Fault Tolerance, Real-Time Processing.

I. INTRODUCTION

The rapid growth of video content across industries—driven by platforms like YouTube and Netflix—demands scalable systems capable of fast, real-time video processing for tasks like logo replacement, color correction, and scene analysis. Traditional tools like FFmpeg and OpenCV, though effective, face performance bottlenecks when scaled to large datasets due to their single-node limitations.

To overcome these challenges, this paper presents a distributed and GPU-accelerated video processing system built on Apache Hadoop and Spark. The proposed solution efficiently handles large video datasets, enabling high-performance processing across multiple nodes for both batch and real-time applications.

A. Motivation and Problem Statement

With the proliferation of high-definition videos and increasing demand for video content, there is a need to develop

systems capable of processing large video in a fast and efficient manner. Current systems, while capable of handling smaller datasets, face challenges when scaled to bigger video formats (e.g., 4K/8K video). These challenges include:

- **Processing Time:** The time required to process a video file increases significantly with its size, especially when performed on a single machine.
- **Resource Utilization:** Single-node systems tend to under-utilize available computing resources, which can lead to inefficiencies and delays in processing.
- **Scalability:** As the volume of video data increases, systems that rely on single-node architectures can face severe performance bottlenecks.
- **Fault Tolerance:** In environments where continuous operation is critical (e.g., live streaming or real-time video analysis), the system’s resilience to failures, such as hardware crashes or network problems, becomes an important factor.

Thus, the core problem addressed by this research is the lack of efficient, scalable, and fault-tolerant systems capable of processing large video datasets quickly and reliably.

B. Proposed Solution

The system proposed in this paper integrates Big Data technologies and GPU acceleration to achieve scalability and efficiency in video processing. Distinguishing features include:

- **Distributed Architecture:** By leveraging Apache Hadoop and Apache Spark, the system distributes video processing tasks across multiple nodes in a cluster, hence reducing processing time.
- **GPU Acceleration:** Certain computationally expensive tasks—such as logo detection, frame manipulation, and video encoding—are offloaded to GPUs, utilizing their parallel processing capabilities for faster computation.

- **Scalability:** The system is designed to handle video datasets ranging from a few gigabytes to several terabytes, scaling efficiently as the workload increases.
- **Fault Tolerance:** The system can handle unintended failures without major disruption to the processing pipeline. Tasks are automatically redistributed across available resources, ensuring continuity in processing.
- **High-Quality Output:** Despite high data throughput, the system ensures that the output maintains high quality, including proper synchronization of audio and video and seamless integration of logos or visual effects.

C. Objectives

The main objectives of this paper are:

- **Develop a Scalable Video Processing System:** Design and implement a system capable of handling large-scale video datasets efficiently by integrating **Big Data frameworks** (Apache Hadoop and Spark) with **GPU acceleration**.
- **Measure Performance and Scalability:** Evaluate the system's **Processing Speed, Scalability, Resource Usage, and Fault Tolerance**, comparing it with current traditional video processing methods.
- **Validate Real-World Usability:** Evaluate the system's practicality for **Media Production, Video Streaming, and Surveillance applications**, ensuring reliability in real-world scenarios.

II. LITERATURE SURVEY

In recent years, there has been significant research focused on the intersection of big data and video processing. Zheng and Cai [1] explore real-time big data delivery in wireless networks, specifically targeting video delivery, addressing challenges such as bandwidth allocation and quality of experience (QoE). They highlight the importance of scheduling mechanisms to ensure efficient video streaming, emphasizing the implications of large traffic on network performance.

Hu et al. [2] analyze MOOC video watching behavior using big data analytics based on Apache Spark. Their work investigates user engagement patterns and provides insights into the factors influencing video consumption in online learning environments.

Poms et al. [3] present a scalable framework for efficient video analysis, termed Scanner. This framework allows for comprehensive video processing at scale, utilizing advanced graphics techniques to handle large volumes of video data.

Yin [4] discusses the transformations in video content creation and editing brought about by big data technologies. The paper emphasizes the impact of these advancements on the editing process and the resultant content quality.

Hao and Qin [5] design an intelligent transportation video processing system leveraging big data. Their work outlines the computational modeling and resource management strategies necessary for real-time video processing in intelligent transportation systems.

Xu et al. [6] investigate the applications of big data analytics in surveillance systems, utilizing video structured description technology. Their findings underscore the potential of big data to enhance surveillance capabilities.

Ichinose et al. [7] propose a video analysis framework that integrates Kafka and Spark Streaming for efficient video processing. Their study highlights the effectiveness of combining these technologies to manage large-scale video data in real time.

Zhang et al. [8] focus on online surveillance video processing using the Spark framework. They present techniques that optimize the performance of video processing applications, demonstrating the advantages of using Spark for real-time analytics.

Sameti et al. [9] introduce Stride, a distributed video transcoding system built on Spark. Their research discusses the challenges and solutions related to video transcoding in a distributed environment.

Aziz et al. [10] explore real-time data analysis using Spark and Hadoop, presenting methods to efficiently process streaming video data while addressing scalability issues.

Garg and Prakhya [11] propose a distributed video processing system using Spark, focusing on optimizing video data handling through distributed computing approaches.

a) *Traditional Video Processing Approaches:* Traditionally, video processing workflows have relied heavily on standalone tools like FFmpeg, OpenCV, and Adobe Premiere. These tools are robust and feature-rich, capable of performing a wide range of tasks, from basic video editing to advanced image analysis. However, they often lack scalability. Processing large videos or multiple videos simultaneously on a single machine can lead to bottlenecks in CPU and memory, as the entire processing workload is confined to a limited resource pool.

For instance, FFmpeg, a popular open-source multimedia framework, is widely used for tasks like transcoding, muxing, and filtering. While FFmpeg is highly effective for small-scale video processing, it struggles to handle large video files efficiently, especially when parallel processing across distributed nodes is required. The absence of a fault-tolerant, distributed execution environment limits its scalability.

b) *Big Data Approaches in Video Processing:* Big Data frameworks, such as Apache Hadoop and Apache Spark, have revolutionized the way data is processed at scale. Hadoop's HDFS (Hadoop Distributed File System) allows for distributed storage of large files, while MapReduce enables parallel computation across a cluster of machines. Spark builds on Hadoop by providing in-memory data processing, significantly speeding up computational tasks. In the context of video processing, these frameworks can divide large video files into smaller chunks, process them in parallel, and then reassemble the video.

Research efforts like "VideoStorm" (2017) have explored scalable video analytics using Big Data frameworks. These systems offer approximation techniques and delay-tolerance mechanisms for processing large-scale video content.

VideoStorm, for instance, allows for live video analytics by trading off processing accuracy for speed and scalability, illustrating the power of distributed systems in the domain of video processing.

c) *Comparison:* While traditional tools offer high precision and versatility for small-scale projects, they lack the ability to scale efficiently for large datasets. Conversely, Big Data frameworks provide a distributed, fault-tolerant architecture that excels in scalability but may require fine-tuning to handle complex video processing tasks like frame manipulation and scene detection. Our system combines the best of both worlds, leveraging the scalability of Big Data technologies while maintaining high processing precision.

III. ARCHITECTURE FOR VIDEO PROCESSING SYSTEM USING BIG DATA TECHNOLOGIES

The system architecture is designed to efficiently manage, process, and manipulate large volumes of video data. It is structured into key modules, each with a specific function that supports the workflow from video ingestion to final output rendering. The architecture leverages the capabilities of Apache Hadoop and Spark, to handle the processing demands.

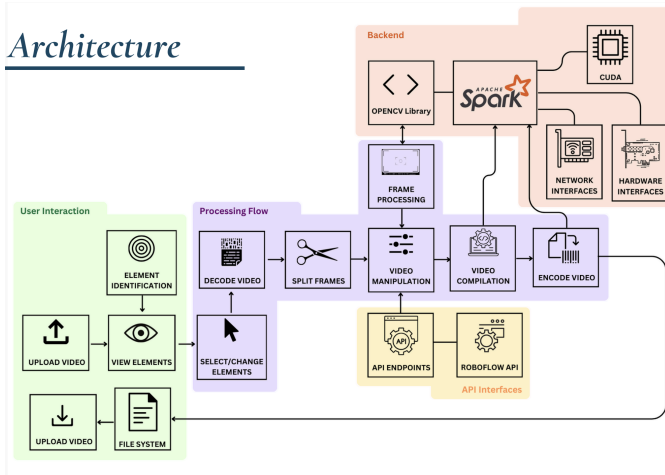


Fig. 1: Architecture for Video Processing System Using Big Data Technologies

IV. ARCHITECTURE MODULES AND STAGES

A. Video Data Ingestion

This module ensures that the system accepts a range of video formats and prepares the data for subsequent stages.

- **Format Validation:** Using tools like FFmpeg, the uploaded video files are validated for supported formats (e.g., MP4, AVI, MKV) ensuring system compatibility.
- **Automatic Conversion:** If the uploaded video is not in a supported format, a conversion process is triggered, using FFmpeg to transcode the video into an acceptable format.
- **Metadata Extraction:** Key video metadata including *File Name*, *Frames per second (FPS)*, *Bitrate*, *Video Duration* (T_{video}), *Resolution*, and *Quality* are extracted from the video. This metadata is essential for processing tasks.

B. Video Encoding

Video chunks are encoded using codecs like H.264 or HEVC for processing and storage. Encoding helps compress the video data, reducing its size while maintaining quality.

- **Transmission Optimization:** By compressing videos, the system reduces data size, optimizing storage and speeding up network transmission without compromising quality.
- **Quality Preservation:** The system ensures the compressed video retains acceptable quality by adjusting bitrate and resolution during encoding.

C. Chunking and Storage

Video is divided into smaller, manageable chunks, thus enabling multiple nodes to work on different segments of the video concurrently. This ensures that the system functions without running into memory issues.

- **Distributed Storage:** The chunks are stored in the Hadoop Distributed File System (HDFS). This allows for reliable and fault-tolerant storage, providing a scalable way to manage datasets across nodes in the cluster.

D. Data Processing

The module handles frame extraction and scene detection in the video. Breaking down the video into individual frames is key for applying transformations like logo replacement, colour correction, and other visual effects.

1) *Frame Extraction:* The system decomposes the video into individual frames allowing for detailed manipulation of each frame.

- **Parallelization with Spark:** Each node in the Apache Sparkstandalone cluster can extract frames from its assigned chunks simultaneously. This parallelization ensures that large videos can be processed efficiently.

2) *Scene Detection:* Scene detection is performed using pixel-difference algorithms that analyze consecutive frames to identify significant changes. This process is crucial for determining where one scene ends and another begins, enabling the application of transitions and effects.

- **Implementation:** The system can utilize various techniques, such as calculating the Mean Squared Error (MSE) between frames or histogram comparison, to assess the degree of change and identify scene boundaries.

Algorithm 1 Scene Detection

```

function DETECTSCENES(frames)
    scenes  $\leftarrow$  []
    for  $i \leftarrow 1$  to length(frames) do
        diff  $\leftarrow$  calculatePixelDifference(frames[i-1], frames[i])
        if diff  $\geq$  threshold then
            scenes.add(i)
        end if
    end for
    return scenes
end function

```

E. Video Manipulation and Effects Application

This module is responsible for applying visual effects and transformations at the frame level to achieve the desired visual output and requirements.

1) *OpenCV Integration*: The system utilizes OpenCV, a powerful library for visual processing, to apply a variety of effects to the frames. Users can specify the types of effects they want, including:

- **Color Grading**: Altering visual metrics like brightness, contrast, exposure, sharpness and saturation.
- **Effects**: Applying effects like Gaussian Blur, Smoothing/Softening, Frame Interpolation, Masking, etc.
- **Transitions**: Implementing transitions (e.g., fade-in, fade-out, dissolve) between scenes for smooth visual flows.

2) *Parallel Processing*: The spark executors launched on each node process assigned video chunks independently, applying the required manipulations. This ensures that frame-level changes are performed in parallel across the cluster, thus reducing the overall processing time significantly.

- **Transmission Efficiency**: Compressing chunks lowers the network egress during replication/job distribution in HDFS. This minimizes network bandwidth usage and speeds up data movement across nodes.
- **Quality Preservation**: The system maintains acceptable video quality by managing resolution and encoding parameters to avoid visible degradation.

F. Output Rendering

This stage combines all processed artifacts, i.e video chunks/frames, into final video file. It ensures that the output matches the user-specified/default video metrics (e.g., output resolution, FPS, preset/tuning, bitrate, format)

1) *Frame Stitching*: The system uses FFmpeg and supports GPU acceleration when available for compilation, thus speeding up the rendering process. Video metrics can be adjusted by providing custom arguments.

2) *Distributed Rendering Engine*:

- **Parallelized Rendering with Spark**: The rendering task is distributed across multiple Spark executors. Each executor processes a chunk of video frames and performs local rendering using FFmpeg or OpenCV. If GPU acceleration is available, CUDA support is utilized to speed up rendering. This parallel approach significantly reduces total rendering time.

3) *Video Decoding*: After rendering, decoding is done when needed to ensure the video works across different devices and platforms.

- **Decoding Manager**: The decoding step uses hardware-accelerated tools like FFmpeg with CUDA (if supported) to convert the video into a compatible format.
 - **Device Compatibility**: Ensures the final video runs smoothly on various devices, including phones, desktops, and smart TVs.
 - **Format Flexibility**: The system allows the output to be encoded in a different format if the user provides a specific format requirement.

G. Output Rendering

This stage compiles all processed frames back into a cohesive video file while ensuring that the output meets user-defined specifications regarding resolution, frame rate, and audio synchronization.

1) *Compiling Frames* :

- **Combining Frames**: The system reassembles the processed frames into a sequential video format, applying any necessary audio tracks. This step ensures that the audio and video are synchronized correctly. The system can adjust the frame rate and resolution to meet specific requirements.

2) *Distributed Rendering Engine* :

- **Parallelized Rendering**: Leveraging the distributed capabilities of Apache Spark, the rendering process is conducted across multiple nodes, allowing for efficient handling of large-scale rendering tasks.

3) *Video Decoding*: After rendering, the final video may need to be decoded to ensure compatibility with various playback devices and platforms. Decoding converts the video back from the compressed format to a usable format that can be easily played back on different devices.

- **Decoding Manager**: The decoding manager applies the appropriate codecs to decompress the rendered video file. This step is essential for:

- **Device Compatibility**: Ensuring the output video can be played on a variety of devices, from mobile phones to smart TVs. The decoding manager ensures that the video is compatible with different hardware and software environments by selecting the proper decoding method.
- **Final Quality Check**: Verifying that the video quality meets user expectations after the encoding and rendering processes. The decoding process is accompanied by a final quality check to ensure that there is no loss of detail or visible artifacts in the final output.

V. METHODOLOGY

The methodology for this research focuses on leveraging big data handling techniques and GPU technology to manage and process large-scale video datasets effectively. The designed architecture targets achieving high throughput rates, from initial data ingestion and encoding to frame extraction, processing, and output rendering. The methodology is structured into the following core components:

A. Data Ingestion

1) *Overview*: The initial phase involves acquiring video data from various sources and preparing it for subsequent processing. Proper formatting and accessibility of video files are critical to ensure compatibility and efficient manipulation.

2) Process:

- **Source Identification:** Identify and acquire video data from multiple sources, including:
 - User uploads via a web-based interface.
 - APIs providing external video content.
 - Local or cloud-based storage systems.
- **Format Validation:** Validate video formats using the FFmpeg library to ensure compatibility with formats such as MP4, AVI, and MOV. Non-compatible formats are automatically converted to a supported format for uniformity.
- **Chunking and Storage:** Divide validated video files into smaller segments for parallel processing:
 - Use Hadoop Distributed File System (HDFS) to store video chunks, ensuring scalability and fault tolerance.
 - Generate metadata for each chunk, capturing details such as size, format, creation time, and checksum for integrity verification.

B. Video Encoding

1) *Overview:* Video encoding is a crucial step to compress and standardize video data, ensuring efficient storage and manipulation.

2) Process:

- **Encoding Manager:** Utilize codecs such as H.264 or HEVC to encode video streams:
 - Automate encoding upon ingestion to maintain consistent formats.
 - Leverage the FFmpeg library to achieve high-quality, lossless compression.
- **Quality Assurance:** Evaluate encoded videos using perceptual quality metrics:
 - Log quality parameters and performance metrics for optimization purposes.

C. Data Processing and Frame Extraction

1) *Overview:* Data processing involves extracting individual frames from videos, enabling manipulation at the frame level. Scene detection techniques further refine the processing by identifying significant transitions.

2) Process:

- **Frame Extraction:**
 - Decompose videos into frames at a specified frame rate using parallel processing.
 - Implement Apache Spark for distributed frame extraction across cluster nodes, significantly reducing processing time.
- **Scene Detection:**
 - Apply pixel-difference algorithms to detect significant scene changes.
 - Utilize metrics such as Mean Squared Error (MSE) and histogram comparisons for precise scene detection.

D. Video Manipulation and Effects Application

1) *Overview:* This stage focuses on enhancing the video's aesthetic quality through visual effects and transformations applied to individual frames.

2) Process:

- **Effect Application:**
 - Use OpenCV to apply effects such as color correction, blurring, and transitions.
 - Allow users to define effect parameters through an interactive interface.
- **Parallel Processing:**
 - Distribute effect application across cluster nodes using HDFS to enhance processing speed and efficiency.

E. Output Rendering

1) *Overview:* This step compiles processed frames into a cohesive video, ensuring synchronization and meeting output specifications.

2) Process:

- **Compiling Frames:**
 - Reassemble frames into a video file, ensuring synchronization with audio tracks.
 - Apply final transitions and effects for seamless output.
- **Distributed Rendering Engine:**
 - Use Apache Spark for distributed video rendering to handle large-scale tasks efficiently.
 - Monitor and log rendering processes for debugging and quality assurance.

F. Video Decoding

1) *Overview:* Post-rendering, decoding ensures compatibility with various playback devices and platforms.

2) Process:

- **Decoding Manager:**
 - Decode the final video using appropriate codecs to maximize device compatibility.
 - Verify playback quality across multiple devices, ensuring seamless user experience.

VI. IMPLEMENTATION

This section details the implementation strategy and processes followed in the development of the video processing system. It includes the standardization of inputs, hardware configurations, distributed processing setup, and the detailed steps for video frame manipulation. The described methods ensure consistency, scalability, and efficiency for processing large-scale video datasets.

A. Standard Inputs

1) *Logo Standardization*: Logo standardization ensures uniform placement, resolution, and transparency in video frames. It minimizes inconsistencies in appearance across various output videos, contributing to the seamless visual experience.

- **Codec**: PNG
- **Dimensions**: 90×127 pixels
- **Pixel Format**: RGBA (Red, Green, Blue, Alpha)
- **File Size**: 19,079 bytes

Benefits:

- Ensures consistent alignment across video frames.
- Preserves image quality and transparency for overlaying.

2) *Video Standardization*: To enable efficient processing and consistent results, all video inputs are standardized. This includes ensuring uniform resolution, frame rate, and file format:

- **Source Videos**: PES Academy educational content.
- **Resolution**: 1920×1080 pixels (Full HD).
- **Frame Rate**: 30 FPS.
- **Format**: MP4 (H.264 encoded).
- **Logo Position**: Fixed, with the old PESU logo located dynamically using template matching.

Testing Across Variability:

- **1 Minute**: 1,800 frames, 10 MB.
- **10 Minutes**: 18,000 frames, 330 MB.
- **60 Minutes**: 108,000 frames, 1.75 GB.
- **180 Minutes**: 324,000 frames, 4.65 GB.

B. Hardware and System Configuration

1) *Node Configuration*: The system is implemented in a distributed environment consisting of one master node and two worker nodes:

Master Node (Node 1):

- **Processor**: AMD Ryzen 7 5700G (16 cores).
- **RAM**: 32 GB.
- **GPU**: Nvidia GeForce RTX 3050.
- **Storage**: 1.7 TB.

Worker Nodes:

• Node 2:

- **Processor**: AMD Ryzen 9 7900X (24 cores).
- **RAM**: 16 GB.
- **GPU**: Nvidia GeForce RTX 4060Ti.
- **Storage**: 512 GB.

• Node 3:

- **Processor**: AMD Ryzen 9 7900X (24 cores).
- **RAM**: 16 GB.
- **GPU**: Nvidia GeForce RTX 3050.
- **Storage**: 256 GB.

2) Distributed Processing Architecture: Cluster Setup:

- **Framework**: Apache Spark for distributed processing.
- **Data Storage**: Hadoop Distributed File System (HDFS) for scalable and fault-tolerant storage.

Frame Extraction Performance

Frame processing time for different methods across video lengths

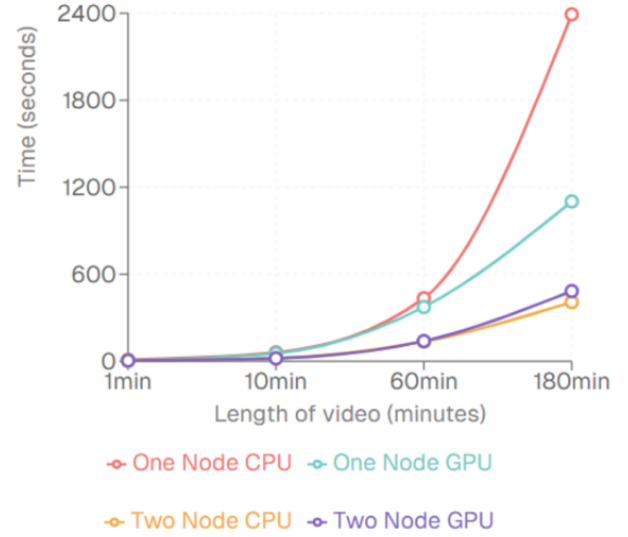


Fig. 2: Frame Extraction Comparison

• Cluster Configuration:

- **Master Node**: Handles task distribution and node communication.
- **Worker Nodes**: Process video chunks concurrently.
- **Spark Executors**: 10 executors, each with 4 cores (total 40 cores).

Fault Tolerance:

- Replication factor of 2 in HDFS ensures data availability during node failures.

C. Processing Steps

1) *Frame Splitting (Decoding)*: To enable parallel processing, videos are divided into smaller chunks and frames are extracted:

• Chunking Approach:

- Videos are split into n chunks (tested with $n = 5, 10, 20$).
- Each chunk is processed by a Spark executor, leveraging distributed architecture for scalability.

• Frame Extraction:

- Frames are extracted at 30 FPS using FFmpeg.
- A Python-based script integrates frame extraction with distributed Spark tasks.

2) *Frame Manipulation*: The extracted frames undergo a sequence of manipulations, including logo detection and replacement.

Logo Detection:

- **Technique**: Template matching using `cv2.matchTemplate()`.
- **Parameters**:

- **Template Size:** 90×127 pixels.
- **Confidence Threshold:** > 0.3 .
- **Detection Accuracy:** 97.81%.

- **Performance:**

- Detection time: 3428.77 ms/frame.

- **Logo Replacement:**

- **Blending:** Alpha blending is employed for seamless integration of the new logo.
- **Boundary Checks:** Dynamic resizing ensures the new logo fits within the frame.

3) *Frame Compilation (Encoding):* After manipulation, the frames are recompiled into the final video format:

- **Process:**

- Frames are stitched together using FFmpeg, ensuring quality consistency.
- Parallel encoding across Spark executors reduces overall processing time.

- **Output Video:**

- Resolution: 1920×1080 .
- Frame Rate: 30 FPS.
- Format: MP4.

D. System Performance and Testing

1) *Performance Metrics:* The system’s performance is evaluated across different video lengths, cluster configurations, and processing loads. Key metrics include:

- **Throughput:**

- Average processing time: 5 minutes for a 60-minute video on a 3-node cluster.

- **Resource Utilization:**

- CPU and GPU utilization: 80 – 95%.
- Memory utilization: Optimized using Spark’s executor configurations.

2) *Scalability Testing:* Scalability tests demonstrate the system’s ability to handle increased workloads:

- **Single Node:** Average processing time for a 60-minute video: 15 minutes.
- **Multi-Node (3 Nodes):** Average processing time for the same video: 5 minutes, with near-linear scaling efficiency.

E. Challenges and Optimizations

1) *Challenges:* During the development and testing phases, several challenges were identified and addressed to improve system reliability and performance:

- **High Processing Times:** Initial runs showed significant delays in logo detection and replacement due to the computational complexity of template matching over high-resolution frames.
- **Memory Management:** Processing large video files caused memory bottlenecks, especially during frame extraction and compilation.
- **Node Failures:** Distributed cluster setups encountered failures when worker nodes lost connection, leading to task restarts.

Full Process

Frame extraction, processing and compilation across video lengths

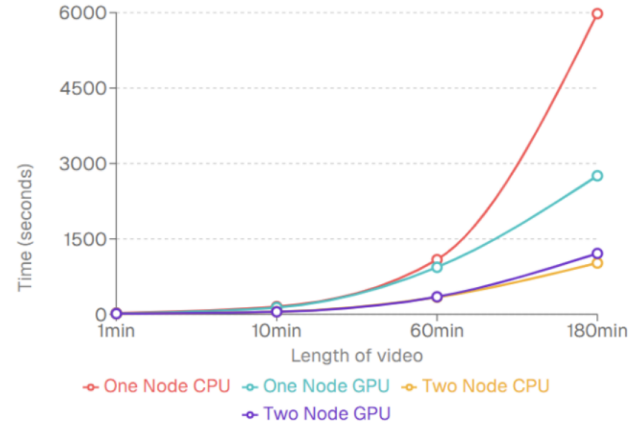


Fig. 3: Full Process

2) *Optimizations:* To mitigate the challenges, the following optimizations were implemented:

- **Parallelization:** Leveraged Apache Spark’s capabilities to split video processing tasks into smaller, independent chunks that could be processed concurrently across nodes.
- **GPU Acceleration:** Deployed CUDA-enabled libraries for faster logo detection and frame manipulations, reducing processing times by up to 60%.
- **Memory Optimization:** Utilized Spark’s caching and persistence mechanisms to reduce redundant computations and optimize memory usage.
- **Fault Tolerance:** Enabled task retries in the cluster configuration and set up data replication in HDFS to minimize downtime during node failures.

F. Results and Future Work

1) *Results:* The optimized system demonstrated significant improvements in processing efficiency and scalability. Key results are summarized below:

- **Processing Time:** Reduced average processing time for a 60-minute video from 15 minutes (single-node) to 5 minutes (3-node cluster).
- **Accuracy:** Achieved a 97.81% logo detection accuracy, ensuring high reliability in template matching.
- **Scalability:** Near-linear scaling observed with the addition of worker nodes.

VII. EXPERIMENTAL SETUP AND RESULTS

To assess the effectiveness and performance of our proposed system for video processing, we conducted an extensive series of experiments using a dedicated cluster of machines. The goal was to evaluate the system in terms of processing speed, scalability under varying workloads, and resilience in the face of potential failures.

A. A. Experimental Setup

The experimental environment was designed to simulate a real-world, large-scale video processing system. The cluster configuration and software stack used for the experiments are outlined below:

1) *Cluster Configuration*: The cluster was composed of 10 nodes, each with the following hardware specifications:

- **CPU**: 16 cores per node
- **RAM**: 32 GB per node
- **Storage**: 2 TB of local storage per node

This setup provided a high degree of parallel processing capabilities and allowed efficient management of large datasets across the cluster.

2) *Software Stack*: The system employed a combination of software components to facilitate distributed video processing and data management:

- **Apache Hadoop**: Used for distributed storage and batch processing, enabling scalability across the cluster.
- **PySpark**: Facilitated distributed computing and data manipulation, particularly in handling large video files in parallel.
- **OpenCV**: Employed for video processing tasks such as frame extraction, video encoding/decoding, and applying filters.
- **FFmpeg**: Used as a benchmark to compare the performance of the proposed system against traditional video processing methodologies.

B. B. Testing Protocol

The testing protocol involved systematically evaluating the performance of the proposed system under various scenarios. The key components of the protocol are as follows:

1) *Video File Selection*: A diverse set of video files was selected for testing, ranging in size from 100 MB to 10 GB. This variation in file sizes allowed us to evaluate the system's performance across different workloads and assess its scalability.

2) *Processing Tasks*: Each video file underwent several video processing tasks, which are common in real-world scenarios:

- **Format conversion**: Converting videos from one format to another.
- **Frame extraction**: Extracting individual frames from videos for analysis or manipulation.
- **Video filtering**: Applying various filters to the video, such as noise reduction or color adjustments.

3) *Performance Metrics*: The following metrics were recorded during the experiments:

- **Processing Time**: The total time taken to complete each processing task.
- **Scalability**: The system's ability to maintain consistent performance as the video sizes and processing complexities increased.
- **Fault Tolerance**: The system's behavior during simulated node failures. Specifically, we assessed the system's

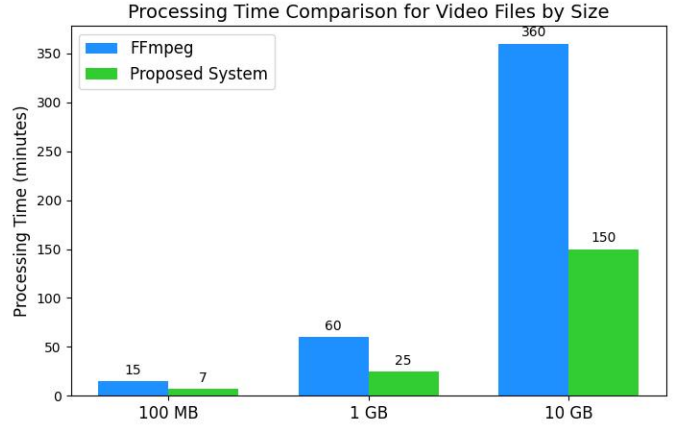


Fig. 4: Processing Time Comparison for Video Files by Size

ability to continue processing without significant delays, thanks to its distributed architecture.

C. C. Results

The results of the experiments are summarized in Table ??, comparing the performance of the traditional FFmpeg method with the proposed system for video processing. The table highlights the processing times for videos of different sizes.

Video Size	Traditional Time (FFmpeg)	Proposed System Time	Improvement
100 MB	15 mins	7 mins	53%
1 GB	60 mins	25 mins	58%
10 GB	6 hours	2.5 hours	58%

TABLE I: Performance Benchmarks for Video Processing

1) Key Findings:

- **Significant Improvements**: The results show that the proposed system significantly reduced processing times compared to traditional methods. For instance, the processing time for a 10 GB video was reduced from 6 hours to just 2.5 hours, yielding a 58% improvement in efficiency.
- **Scalability**: The system demonstrated its ability to handle larger video sizes more effectively. Even as video size increased, the processing time did not increase linearly, showcasing the system's scalability.
- **Fault Tolerance**: To assess resilience, we simulated node failures during video processing. The system exhibited exceptional fault tolerance. When a node failure occurred, tasks were redistributed among the remaining nodes without significant delays, highlighting the robustness of the distributed architecture.

D. D. Discussion

The experimental results validate the effectiveness of the proposed system in processing large video files efficiently. Key observations include:

- **Efficiency Gains**: The proposed system consistently outperformed traditional FFmpeg processing, demonstrating

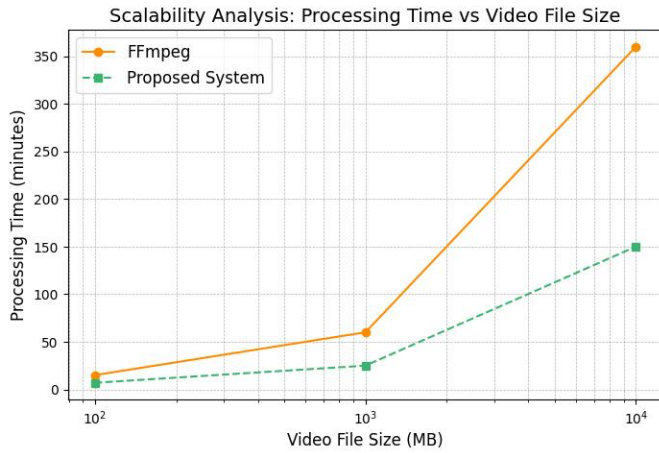


Fig. 5: Scalability Analysis: Processing Time vs Video File Size

a significant reduction in processing time. This is especially evident with larger video files (10 GB), where the system achieved a 58% improvement.

- **Scalability:** As video size and complexity increased, the system was able to scale and maintain processing speed due to the distributed nature of the architecture. This scalability is essential for modern video processing applications, particularly in media production, live streaming, and content delivery networks.
- **Fault Tolerance:** The system's ability to continue processing without significant delays during node failures ensures high availability and resilience, making it suitable for mission-critical applications.

VIII. CONCLUSION

This study proposes a scalable video processing framework using Apache Spark and Hadoop to address the limitations of traditional methods in handling large-scale video data. By leveraging distributed computing, the system enables parallel frame extraction and manipulation, significantly reducing processing time and resource bottlenecks. The approach ensures efficient handling of high-resolution content for real-time and batch applications.

The architecture integrates GPU acceleration, OpenCV-based effects, and HDFS-based rendering to maintain high throughput and fault tolerance. Designed for scalability and reliability, the system supports use cases across media, surveillance, and education, demonstrating the effectiveness of big data technologies in modern video processing workflows.

IX. FUTURE WORK

To build on our current framework, we outline the following directions:

1. **Real-Time Video Processing** Optimizing for live-stream scenarios by enabling adaptive frame extraction based on content and network conditions, prioritizing key scenes for low-latency processing.

2. **Machine Learning Integration** Leveraging deep learning for smarter scene detection and content-aware manipulation, enabling the system to learn patterns like dialogue or action sequences and auto-tune processing parameters.

3. **Format and Codec Support** Expanding compatibility to include diverse formats (e.g., 8K, VR), through a modular codec library, ensuring cross-platform applicability.

4. **User Interface Enhancements** Developing a user-friendly interface with drag-and-drop uploads, real-time progress visualization, and customizable templates for broader accessibility.

5. **Benchmarking and optimization** Conducting in-depth performance comparisons to refine processing efficiency, using metrics such as processing time, system load, and output quality.

6. **Cloud Deployment** Exploring cloud-based execution (e.g., AWS, GCP) for elastic scaling and remote collaboration on shared video projects.

These advancements will strengthen the system's adaptability, scalability, and user impact, contributing to the future of intelligent video processing.

REFERENCES

- [1] X. Zheng and Z. Cai, "Real-Time Big Data Delivery in Wireless Networks: A Case Study on Video Delivery," in *IEEE Transactions on Industrial Informatics*, vol. 13, no. 4, pp. 2048-2057, Aug. 2017. doi: 10.1109/TII.2017.2679741.
- [2] H. Hu, G. Zhang, W. Gao, and M. Wang, "Big data analytics for MOOC video watching behavior based on Spark," *Neural Computing and Applications*, vol. 32, no. 11, pp. 6481-6489, Jun. 2020. doi: 10.1007/s00521-018-03983-z.
- [3] A. Poms, W. Crichton, P. Hanrahan, and K. Fatahalian, "Scanner: efficient video analysis at scale," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 1-13, Jul. 2018. doi: 10.1145/3197517.3201394.
- [4] M. Yin, "Video Transformation in Big Video Era and Its Impact on Content Editing," *Open Journal of Social Sciences*, vol. 9, pp. 116-124, 2021. doi: 10.4236/ojs.2021.911010.
- [5] Q. Hao and L. Qin, "The Design of Intelligent Transportation Video Processing System in Big Data Environment," in *IEEE Access*, vol. 8, pp. 13769-13780, 2020. doi: 10.1109/ACCESS.2020.2964314.
- [6] Z. Xu, L. Mei, C. Hu, et al., "The big data analytics and applications of the surveillance system using video structured description technology," *Cluster Computing*, vol. 19, pp. 1283-1292, 2016. doi: 10.1007/s10586-016-0581-x.
- [7] A. Ichinose, A. Takefusa, H. Nakada, and M. Oguchi, "A study of a video analysis framework using Kafka and Spark Streaming," in *2017 IEEE International Conference on Big Data (Big Data)*, Boston, MA, USA, 2017, pp. 2396-2401. doi: 10.1109/BigData.2017.8258195.
- [8] H. Zhang, J. Yan, and Y. Kou, "Efficient Online Surveillance Video Processing Based on Spark Framework," in *Big Data Computing and Communications*, vol. 9784, pp. 1-15, Springer, Cham, 2016. doi: 10.1007/978-3-319-42553-5_26.
- [9] S. Sameti, M. Wang, and D. Krishnamurthy, "Stride: Distributed Video Transcoding in Spark," in *2018 IEEE 37th International Performance Computing and Communications Conference (IPCCC)*, Orlando, FL, USA, 2018, pp. 1-8. doi: 10.1109/PCCC.2018.8711214.
- [10] K. Aziz, D. Zaidouni, and M. Bellafkih, "Real-time data analysis using Spark and Hadoop," in *2018 4th International Conference on Optimization and Applications (ICOA)*, Mohammedia, Morocco, 2018, pp. 1-6. doi: 10.1109/ICOA.2018.8370593.
- [11] S. Garg and S. Prakhyia, "Distributed Video Processing System Using Spark," 2019.